OS GTU SUPER 40 QUE ANS

Q1. What is Operating System? Explain Fundamental Goals of OS

Operating System (OS):
An OS is system software that works as a link between the user and computer hardware. It manages CPU, memory, files, and I/O devices and provides a platform for running applications.
Examples: Windows, Linux, Android.

Fundamental Goals of OS

1. Convenience – Makes computer easy to use by providing interfaces like GUI or CLI.

2. Efficiency – Uses CPU, memory, and devices in the best possible way.

3. Ability to Evolve – Can be updated or improved without affecting existing programs.

4. Resource Management – Manages CPU, memory, I/O devices, and files.

5. Security – Protects system data and resources using passwords, permissions.

6. Error Handling – Detects and solves runtime hardware/software errors.

7. Multitasking – Allows many programs to run at the same time.

Q2. What is OS? Explain Need of Operating System

Operating System:
It is system software that controls hardware and runs application programs.

Need of OS

1. Hardware Management – Controls CPU, memory, and I/O devices.
2. Program Execution – Loads and runs applications.
3. User Interface – Provides GUI or CLI for easy interaction.
4. Memory Management – Allocates and tracks memory for processes.
5. Process Management – Starts, stops, and schedules processes.
6. File Management – Creates, deletes, and organizes files.
7. I/O Handling – Manages devices like keyboard, printer, disk.
8. Security – Prevents unauthorized access.
9. Error Handling – Detects and manages system errors.
10. Resource Allocation – Fairly allocates CPU, memory, and devices.

Q3. Explain Operating System Services

OS provides essential services to users and programs:

1. Program Execution – Loads and executes programs.
2. I/O Operations – Handles input and output devices.
3. File System Management – Manages files and folders.
4. Communication – Allows processes to exchange data.
5. Error Detection – Detects and reports hardware/software errors.
6. Resource Allocation – Assigns CPU, memory, and I/O devices.
7. Protection & Security – Controls access to system resources.
8. Accounting – Monitors resource usage.
9. User Interface – Provides CLI/GUI to interact with system.

Q4. Define OS and Explain Components of Computer System

Operating System

Software that manages hardware and provides services for applications. It acts as a link between user and hardware.

Components of Computer System

1. Hardware – Physical parts: CPU, RAM, disk, keyboard, monitor.
2. Operating System – Controls hardware and runs programs.
3. Application Programs – Software for user tasks (Word, browser, games).
4. Users – People who operate the computer.

Flow:
Users → Applications → Operating System → Hardware

Q5. Types of OS. Explain Batch OS with Advantages & Disadvantages

Types of Operating Systems

Batch OS
Time-sharing OS
Multiprogramming OS
Real-Time OS
Distributed OS
Network OS
Multithreading OS
Mobile OS

Batch Operating System

Meaning:
Executes similar jobs in batches without user interaction.

How it Works:

Jobs → Grouped into batch → Executed one after another → Output returned later.

Advantages:

1. High CPU utilization
2. Good for large repetitive jobs
3. No user involvement needed
4. Faster execution for batch jobs

Disadvantages:

1. Long waiting time
2. No interaction during execution
3. Error affects the whole batch
4. Needs job preparation beforehand

Q6. Short Note on Multiprogramming OS and Multithreading OS

1. Multiprogramming OS

Multiple programs stay in memory together.
CPU switches when one program waits for I/O.
Improves CPU utilization and throughput.

Advantages:
High CPU use, better performance

Disadvantages:
Complex memory management

2. Multithreading OS

A single program is divided into multiple threads.
Threads share memory but run independently.

Advantages:
Faster execution, responsive applications

Disadvantages:
Synchronization issues, debugging difficulty

Q7. Types of OS. Explain Time-Sharing & Real-Time OS

Time-Sharing OS

CPU time divided among multiple users.
Each user gets a small time slice.

Advantages:
Quick response, efficient CPU use

Disadvantages:
Security issues, performance drops under heavy load

Real-Time OS (RTOS)

Provides immediate response to events.
Used in critical systems.

Types:

1. Hard RTOS – No delay allowed (airbags).
2. Soft RTOS – Small delay allowed (video streaming).

Advantages:
Reliable, accurate, fast

Disadvantages:
Expensive, complex design

Q8. Difference Between Program and Process

Feature	Program	Process
Meaning	Set of instructions	Program in execution
State	Passive	Active
Resources	No resources until run	Uses CPU, memory, I/O
Execution	Not executed by itself	Executing on CPU
Count	One program → many processes	Each process is one instance

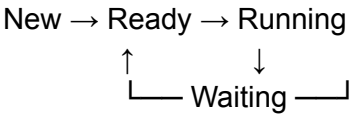
Q9. Explain Process Life Cycle
A process goes through different states from start to end.

Process States

1. New – Process created
2. Ready – Waiting for CPU
3. Running – Executing on CPU

4. Waiting/Blocked – Waiting for I/O or event
5. Terminated – Finished execution

Text Diagram



Key Points

OS switches process states using scheduling.
Helps in multitasking and CPU efficiency.

Q10. What is Thread? Explain Benefits of Using Threads
Thread

A thread is the smallest unit of execution inside a process.
A process may have multiple threads that run independently but share the same memory.

Example:
A browser uses different threads for loading page, playing video, and downloading.

Benefits of Threads

1. Faster Execution – Tasks run in parallel.
2. Better CPU Utilization – Idle time is reduced.
3. Easy Communication – Threads share memory of the same process.
4. Improved Responsiveness – Application does not freeze during long tasks.
5. Lower Cost – Creating threads is cheaper than creating processes.
6. Efficient Resource Sharing – No need for separate memory for each thread.

Q11. Explain Process Control Block (PCB) in Detail
PCB is a data structure used by the OS to store all information about a process.

Contents of PCB

1. Process ID (PID) – Unique number for each process.
2. Process State – Ready, Running, Blocked, etc.
3. CPU Registers – Contents of registers when process is stopped.
4. Program Counter (PC) – Address of next instruction.
5. Memory Information – Base/limit registers, page tables.
6. I/O Information – List of I/O devices assigned.
7. Scheduling Information – Priority, CPU time used.

8. Accounting Information – Time limits, user ID.

.Q12. Explain Long-Term, Medium-Term, and Short-Term Schedulers

1. Long-Term Scheduler (Job Scheduler)

Selects which jobs should enter the ready queue.
Controls degree of multiprogramming.
Runs rarely.

2. Medium-Term Scheduler

Removes processes from memory (swapping).
Brings them back later.
Helps manage memory efficiently.

3. Short-Term Scheduler (CPU Scheduler)

Selects which ready process should run on CPU next.
Runs very frequently.
Responsible for fast process switching.

Q13. Explain Mutual Exclusion (Critical Section Problem)

Mutual exclusion means only one process can enter the critical section at a time.

Critical Section

A part of program where shared data is accessed.

Why Needed?

To avoid data corruption
To prevent race conditions

Requirements of Mutual Exclusion

1. Mutual Exclusion – Only one process enters critical section.
2. Progress – If no one is inside, one should be allowed to enter.
3. Bounded Waiting – No process should wait forever.

Methods

Locks
Semaphores
Mutex

Monitors

Q14. Describe Race Condition

Race condition occurs when two or more processes access and modify shared data at the same time, and the final result depends on who executes first.

Example

Two processes withdraw money from the same bank account at the same time → Incorrect final balance.

Why Race Conditions Occur?

Lack of synchronization
Multiple processes running in parallel
No mutual exclusion

Solution

Use locks, semaphores, and critical section techniques.

Q15. Difference Between CPU-Bound and I/O-Bound Process

Feature	CPU-Bound Process	I/O-Bound Process
Depends On	CPU speed	Input/Output devices
Time Spent	More time in calculations	More time waiting for I/O
Example	Video rendering, algebra solving	Reading file, printing
CPU Burst	Long	Short
Performance	Needs fast CPU	Needs fast I/O

Q16. Define Throughput, Turnaround Time, Waiting Time, Response Time, CPU Utilization

Throughput
No. of processes completed per unit time.

Turnaround Time
Time from process submission to completion.
(Completion Time – Arrival Time)

Waiting Time
Total time a process spends in ready queue.
(Turnaround Time – Burst Time)

Response Time

Time from process submission to first response.

CPU Utilization
Percentage of time CPU is busy. Higher is better.

Q17. What is Deadlock? Explain Four Necessary Conditions
Deadlock

A situation where two or more processes are blocked forever because each is waiting for resources held by the other.

Four Conditions

1. Mutual Exclusion – Resources cannot be shared.
2. Hold and Wait – A process holds one resource and waits for another.
3. No Preemption – Resources cannot be forcibly taken away.
4. Circular Wait – Each process waits for a resource held by the next process in a circle.

If all these four conditions exist → Deadlock occurs.

Q18. Deadlock Characteristics + Prevention + Avoidance
Characteristics of Deadlock

Circular waiting
No progress
Resources not released
All processes stuck

Deadlock Prevention

Break any one deadlock condition:
Remove mutual exclusion
Prevent hold & wait
Allow preemption
Break circular wait
Example: Resource ordering

Deadlock Avoidance

OS checks before allocating resources.
Uses Banker's Algorithm to ensure system stays safe.

Q19. Difference Between Logical Address and Physical Address

Logical Address	Physical Address
Generated by CPU	Seen by memory hardware
Virtual address	Real address in RAM
Used by programmer	Used by OS & memory unit
Same for each process	Different for each process

Q20. What is Swapping? Explain with Example

Swapping

Swapping is a memory management technique where a process is moved from RAM to disk (swap space) to free memory.
Later it is brought back to RAM when needed.

Example

RAM is full
OS swaps out Process P1 to disk
Loads Process P2 into RAM
When P1 is needed again → Swap P2 out → Bring P1 back

Benefits

Increases multiprogramming
Efficient use of memory

Disadvantages

Slow due to disk access
Swapping too much causes “thrashing”

Q21. Explain Multiprogramming with Fixed (Static) and Dynamic Partitions

Multiprogramming is a technique used by an operating system in which several programs are kept in main memory at the same time. The CPU executes one program while others are waiting for I/O, which increases CPU utilization and system throughput.

Multiprogramming with Fixed (Static) Partitions

In fixed partitioning, main memory is divided into a fixed number of partitions before execution. Each partition can store only one process.

Advantages:

Simple to design and manage.
Low memory management overhead.

Disadvantages:

Internal fragmentation occurs.
Limits the maximum size and number of processes.

Multiprogramming with Dynamic Partitions

In dynamic partitioning, memory is divided at runtime according to the size of incoming processes.

Advantages:

Reduces internal fragmentation.
Supports processes of different sizes efficiently.

Disadvantages:

External fragmentation may occur.
Memory management is more complex.

Q22. Short Note on Fragmentation

Fragmentation is a problem in memory management where some part of memory is wasted and not used properly.

There are two main types of fragmentation:

1) Internal Fragmentation

It happens when a process gets more memory than it actually needs. The extra memory inside the allocated block remains unused.

Example:
A process needs 40 KB but is given 60 KB. The extra 20 KB is wasted.

2) External Fragmentation

It happens when free memory is available but spread in small pieces. Even if enough total memory is free, it cannot be given to a process in one continuous block.

Q23. Difference Between External Fragmentation and Internal Fragmentation

Basis	Internal Fragmentation	External Fragmentation
Meaning	Wastage of memory inside an allocated memory block	Wastage of memory outside allocated blocks
Cause	Allocated memory block is larger than required	Free memory is broken into small, non-contiguous blocks
Occurs In	Fixed partitioning	Dynamic partitioning
Memory Status	Unused space remains within the allocated block	Enough total free memory exists but not in one continuous block
Example	Process needs 30 KB but gets 50 KB	Total free memory is 100 KB but scattered in small parts
Solution	Use variable-sized partitions	Use compaction or paging

Q24. Short Note on Memory Relocation and Protection

Memory Relocation

Memory relocation is a technique used by the operating system to load a program into any available location in main memory. The program does not need to know its actual physical memory address. Address mapping is done using relocation registers.

Importance:

Allows programs to be moved in memory.
Supports multiprogramming.
Improves memory utilization.

Memory Protection

Memory protection ensures that a process can access only its own allocated memory area. It prevents one process from interfering with the memory of another process or the operating system.

Importance:

Provides system security.
Prevents accidental or illegal memory access.
Increases system stability.

Q26. Explain Address Translation in Paging

Address Translation in Paging is the process of converting a logical (virtual) address generated by the CPU into a physical address in main memory.

In paging, a process's logical address space is divided into fixed-size pages, and physical memory is divided into fixed-size frames of the same size.

Translation Process:

1. The CPU generates a logical address.
2. The logical address is divided into two parts:

Page Number (p)
Page Offset (d)

3. The page number is used as an index in the page table.
4. The page table gives the corresponding frame number.
5. The physical address is formed by combining the frame number with the page offset.

Example:
If page size is 1 KB, the offset identifies the exact location within the frame, while the page number identifies which frame is used.

Q27. What is Paging? Explain TLB with Diagram
What is Paging?

Paging is a memory management technique in which the logical memory of a process is divided into fixed-size blocks called pages, and physical memory is divided into fixed-size blocks called frames.
Pages can be stored in any free frame, so paging removes external fragmentation.

Advantages of Paging:

No external fragmentation
Easy memory allocation
Supports virtual memory

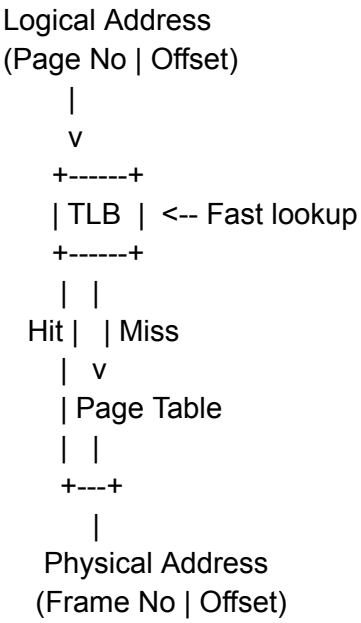
Translation Lookaside Buffer (TLB)

A TLB (Translation Lookaside Buffer) is a small, fast cache memory that stores recent page table entries. It helps in speeding up the address translation process.

Working of TLB:

1. CPU generates a logical address (Page No + Offset).
2. The page number is first searched in the TLB.
3. If found (TLB hit), frame number is obtained quickly.
4. If not found (TLB miss), the page table in main memory is accessed.
5. Physical address is formed using frame number and offset.

Diagram (Address Translation with TLB)



Q28. What is Segmentation? Explain with example.

Segmentation

A memory management technique where memory is divided into variable-sized segments based on program structure.

Common segments:

1. Code segment
2. Data segment
3. Stack segment

Example

If a program has:

Code = 10 KB
Data = 5 KB
Stack = 7 KB

Memory is divided accordingly into 3 separate segments.
Segments are based on logical divisions, not fixed sizes like paging.

Q29. Explain Address Translation in Segmentation

Address Translation in Segmentation is the process of converting a logical address into a physical address using segments.

In segmentation, a program is divided into variable-size logical units called segments (such as code, data, stack). Each segment has a base address and a limit stored in the segment table.

Translation Process:

1. The CPU generates a logical address.
2. The logical address consists of two parts:

Segment Number (s)
Offset (d)

3. The segment number is used to access the segment table.
4. The segment table provides the base address and limit of the segment.
5. The offset is checked against the limit to ensure protection.
6. If valid, the physical address is obtained by adding the base address and offset.

Q30. Explain Various Attributes of a File

A file is a collection of related information stored on secondary storage. The operating system maintains several file attributes to manage and protect files properly.

Various Attributes of a File:

1. File Name
The name used to identify the file in the file system.
2. File Type
Indicates the type of file, such as text file, executable file, or image file.
3. Location
Specifies the physical location of the file on the storage device.
4. Size
Shows the current size of the file in bytes.
5. Protection
Defines access rights like read, write, and execute permissions.
6. Time and Date
Stores information about file creation, last modification, and last access time.
7. Owner/User ID
Identifies the owner of the file and helps in access control.

Q31. Explain various file operations in detail.

The Operating System allows different operations on files. Important file operations are:

1. Create

Used to make a new empty file.

OS allocates space and updates the directory.

2. Open

Before using a file, it must be opened.
OS loads file information into memory.

3. Read

Used to get data from a file.
Example: reading text from a document.

4. Write

Used to store data in a file.
Example: saving text, images, etc.

5. Reposition / Seek

Moves the file pointer to a specific location inside the file.

6. Delete

Removes file permanently from storage.

7. Close

Releases resources used by the file.
Updates any changes made.

8. Append

Add new data to the end of the file without deleting old data.

Q32. Various File Types with One-Line Definition and Extension

Text File (.txt): Stores plain text without formatting.

Document File (.doc, .docx, .pdf): Used to create and store formatted documents.

Spreadsheet File (.xls, .xlsx): Used to store data in rows and columns and perform calculations.

Presentation File (.ppt, .pptx): Used to create slide-based presentations.

Image File (.jpg, .png, .gif): Stores pictures and graphics.

Audio File (.mp3, .wav): Stores sound or music data.

Video File (.mp4, .avi): Stores moving images with sound.

Executable File (.exe): Used to run a program or application.

Compressed File (.zip, .rar): Used to reduce file size by compression.

Source Code File (.c, .cpp, .java, .py): Contains program instructions written in a programming language.

Q33. Explain physical structure of Hard Disk.

A hard disk has the following physical components:

1. Platters

Circular disks that store data magnetically.
Usually made of aluminum or glass.

2. Tracks

Concentric circles on a platter surface.

3. Sectors

A track is divided into small parts called sectors.
Smallest unit of storage (usually 512 bytes or 4 KB).

4. Cylinders

Set of tracks vertically aligned across platters.

5. Read/Write Heads

Tiny devices that read or write data on platters.

6. Spindle

Rotates the platters at high speed (5400–7200 RPM).

7. Actuator Arm

Moves the read/write heads across the platters.

Q34. Write a short note on Disk Space Allocation Methods.

Three main methods are used:

1. Contiguous Allocation

Files stored in continuous memory blocks. Advantages: Fast access.
Disadvantages: Causes external fragmentation.

2. Linked Allocation

File blocks stored anywhere; each block contains a pointer to the next block. Advantages: No fragmentation.
Disadvantages: Slow access, pointer overhead.

3. Indexed Allocation

An index block contains a table of all file block addresses. Advantages: Fast direct access.
Disadvantages: Extra index block required.

Q36. Explain features of Linux operating system.

Main Features

1. Open Source – Free to use and modify.
2. Multitasking – Runs multiple tasks at the same time.
3. Multi-User – Many users can access system simultaneously.
4. Security – Strong permission system.
5. Portability – Runs on many hardware platforms.
6. Networking Support – Powerful tools for networking.
7. Stability and Reliability – Rarely crashes.
8. Shell Support – Command-line interface for advanced tasks.

Q37. Linux Architecture / Components of Linux Operating System

What is Linux Architecture?

Linux architecture is the structure of the operating system that shows how its components interact.
Linux is a monolithic kernel system, which means all core services run in kernel space, but it also supports modular design.

Components of Linux OS

Linux OS has mainly three layers:

A. User Space (User Mode)

This is the top layer where users interact with the system.

Components:

1. User Applications

Programs like web browsers, office software, editors.
Run in user mode, cannot directly access hardware.

2. Shell / Command Line Interface (CLI)

Provides interface to communicate with kernel using commands.
Examples: bash, zsh.

3. Graphical User Interface (GUI)

Desktop environments like GNOME, KDE.
Makes Linux user-friendly.

B. Kernel Space (Kernel Mode)

Kernel is the core of Linux OS, responsible for managing hardware and system resources.

Key responsibilities of Kernel:

1. Process Management

Handles creation, scheduling, and termination of processes.
Supports multitasking and multithreading.

2. Memory Management

Manages RAM allocation, paging, segmentation, and swapping.
Keeps processes isolated and protected.

3. Device Drivers

Kernel interacts with hardware devices through drivers.
Examples: Hard disk, printer, network card, display.

4. File System Management

Manages file storage, retrieval, and protection.
Supports different file systems: ext4, XFS, FAT32, NTFS.

5. System Calls

Interface for user programs to request services from the kernel.
Examples: open(), read(), write(), fork().

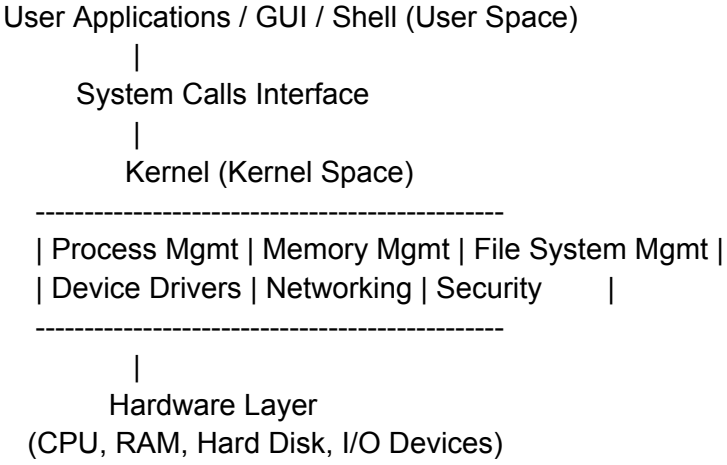
6. Networking

Manages network connections, protocols, and communication.
Supports TCP/IP, UDP, HTTP, FTP, SSH.

C. Hardware Layer

The lowest layer of Linux architecture.
Includes CPU, RAM, Hard Disk, I/O devices, network cards.
Kernel communicates with hardware using device drivers.

Diagram of Linux Architecture (Text Form)



Q38. Explain Linux Commands

Linux commands are instructions given to the operating system to perform specific tasks. Below are commonly used Linux commands explained in easy English.

1. ls Command

Purpose: Lists files and directories in a directory.
Syntax: ls [options] [directory_name]

Examples:
1. ls → Lists files in the current directory
2. ls -l → Shows detailed information (permissions, owner, size, date)
3. ls -a → Shows all files, including hidden files starting with .

2. chmod Command

Purpose: Changes file or directory permissions.
Permissions: Read (r), Write (w), Execute (x)
Syntax: chmod [options] mode filename

Examples:

1. chmod 755 file.txt → Sets permissions to owner: rwx, group: r-x, others: r-x
2. chmod +x script.sh → Adds execute permission to the file

3. wc Command

Purpose: Counts words, lines, and characters in a file.
Syntax: wc [options] filename

Options:

1. -l → Number of lines
2. -w → Number of words
3. -c → Number of bytes/characters

Example: wc -l file.txt → Counts lines in file.txt

4. cat Command

Purpose: Displays the contents of a file or concatenates files.
Syntax: cat [options] filename

Examples:

1. cat file.txt → Shows contents of file.txt on the screen
2. cat file1.txt file2.txt > file3.txt → Combines two files into a new file

5. grep Command

Purpose: Searches for a specific pattern or text in a file.
Syntax: grep [options] pattern filename

Examples:

1. grep "hello" file.txt → Finds all lines containing "hello"
2. grep -i "hello" file.txt → Case-insensitive search

6. sort Command

Purpose: Sorts lines in a file alphabetically or numerically.
Syntax: sort [options] filename

Examples:

1. sort file.txt → Sorts lines alphabetically
2. sort -n file.txt → Sorts numerically

7. rm Command

Purpose: Deletes files or directories.
Syntax: rm [options] filename

Examples:

1. rm file.txt → Deletes file.txt
2. rm -r folder → Deletes directory and all its contents

10. echo Command

Purpose: Displays a message or variable value on the screen.
Syntax: echo [text or $variable]

Example:

1. echo "Hello World" → Prints Hello World
2. echo $HOME → Shows the home directory path

Q39. Shell script to find maximum of three numbers.

```
echo "Enter three numbers:"
read a b c
if [ $a -ge $b ] && [ $a -ge $c ]
then
   echo "Maximum = $a"
elif [ $b -ge $a ] && [ $b -ge $c ]
```

```
then
   echo "Maximum = $b"
else
   echo "Maximum = $c"
fi
```

Q40. Shell script to find factorial of a number.

```
echo "Enter a number:"
read n
fact=1
for (( i=1; i<=n; i++ ))
do
   fact=$((fact * i))
done
echo "Factorial = $fact"
```

Q41. Shell script to print Fibonacci series.

```
echo "Enter number of terms:"
read n
a=0
b=1
echo "Fibonacci Series:"
for (( i=1; i<=n; i++ ))
do
   echo -n "$a "
   c=$((a + b))
   a=$b
   b=$c
done
```

Q42. Shell script to find sum and average of five numbers.

```
echo "Enter five numbers:"
read a b c d e
sum=$((a + b + c + d + e))
avg=$((sum / 5))
echo "Sum = $sum"
echo "Average = $avg"
```